Engenharia Reversa — Oficina AutoCerta

Relatório técnico do sistema web recebido sem documentação

1. Visão geral

Aplicação para autenticação e gestão de clientes, veículos e agendamentos de oficina. Front-end HTML/CSS/JavaScript, back-end Node.js + Express e banco SQLite usando node:sqlite.

2. Requisitos funcionais e regras de negócio

• RF01 — O sistema deve permitir login com usuário e senha.

• RF02 — O sistema deve manter sessão autenticada e permitir consultar o usuário atual.

• RF03 — O sistema deve permitir encerrar a sessão.

• RF04 — O sistema deve listar clientes e pesquisar por nome, CPF, telefone ou e-mail.

• RF05 — O sistema deve cadastrar clientes com nome e CPF obrigatórios, CPF de 11 dígitos e e-mail validado quando informado.

• RF06 — O sistema deve permitir editar dados de clientes existentes.

• RF07 — O sistema deve impedir CPF duplicado entre clientes.

• RF08 — O sistema deve listar veículos com dados do cliente associado.

• RF09 — O sistema deve cadastrar veículos com placa, modelo, tipo e cliente existente.

• RF10 — O sistema deve aceitar placa antiga e Mercosul e impedir placas duplicadas.

• RF11 — O sistema deve listar agendamentos com dados de veículo e cliente.

• RF12 — O sistema deve cadastrar agendamento com veículo existente, data válida e serviço.

• RF13 — O sistema deve permitir alterar somente a data de um agendamento.

• RF14 — O sistema deve retornar erros HTTP e mensagens JSON para entradas inválidas e recursos inexistentes.

• RF15 — O sistema deve armazenar senha com hash bcrypt e CPF criptografado.

• RN01 — Login e senha são obrigatórios; login é comparado após `trim()`, senha é verificada com `bcrypt.compareSync`.

• RN02 — Rotas de dados exigem `req.session.usuario`; ausência resulta em HTTP 401.

• RN03 — CPF é normalizado para dígitos antes de validar e armazenar.

• RN04 — CPF deve conter 11 dígitos; não há validação matemática dos dígitos verificadores.

• RN05 — CPF deve ser único, verificado por descriptografia em memória.

• RN06 — Placa é normalizada para maiúsculas e remoção de espaços/hífens; regex `^[A-Z]{3}\d[A-Z0-9]\d{2}$`.

• RN07 — Modelo e tipo do veículo são obrigatórios; cliente precisa existir.

• RN08 — Agendamento exige veículo existente, data real no formato `AAAA-MM-DD` e serviço não vazio.

• RN09 — Editar agendamento altera apenas a data, não veículo nem serviço.

• RN10 — Não existe regra que impeça agendamentos passados, conflitos de horário ou duplicidade de agendamento.

• RN11 — O sistema não implementa autorização por perfil; qualquer usuário autenticado tem acesso às rotas disponíveis.

• RN12 — O banco SQLite usa chaves estrangeiras, mas a integridade depende de `PRAGMA foreign_keys = ON` na conexão.

• **usuarios** (`id`, `nome`, `login`, `senha`): `login` é único; `senha` guarda hash bcrypt.

• **clientes** (`id`, `nome`, `cpf`, `telefone`, `email`): `cpf` guarda conteúdo cifrado.

• **veiculos** (`id`, `placa`, `modelo`, `tipo`, `marca`, `cliente_id`): placa única; FK para clientes.

• **agendamentos** (`id`, `veiculo_id`, `data_agendamento`, `servico`): FK para veículos.

Controles existentes

• **Sessão:** `express-session`, duração de cookie de 30 minutos (`maxAge`), `httpOnly: true`. Não há `secure`, `sameSite` explícitos, rotação do ID após login nem store persistente configurado; o padrão MemoryStore não é adequado à produção.

• **Hash de senha:** `bcryptjs` e `bcrypt.compareSync`; hashes seed indicam custo 08. O código não cria/atualiza usuários e não contém fluxo de troca de senha.

• **Criptografia do CPF:** AES-256-GCM, chave de 32 bytes derivada via `scryptSync`, IV aleatório de 12 bytes e tag de autenticação. Porém, chave e salt estão fixos no código; isso não equivale a gestão segura de segredos. O banco contém CPFs cifrados.

• **SQL injection:** consultas usam parâmetros `?` nos valores dinâmicos observados.

• **XSS:** front-end aplica `esc()` para os valores renderizados via `innerHTML`; avisos usam `textContent`.

Riscos e limitações

Três melhorias prioritárias

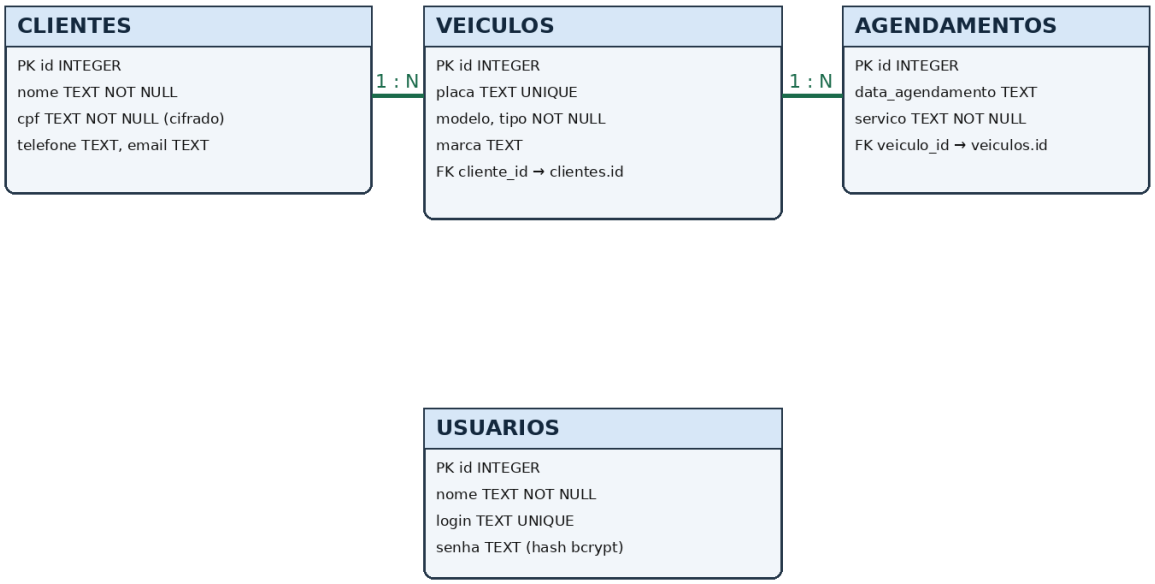
• **M1 — Segredos e sessão:** ler `SESSION_SECRET` e chave de criptografia de variáveis de ambiente/secret manager; rotacionar o ID da sessão após login; configurar cookie `httpOnly`, `secure` em HTTPS e `sameSite: 'lax'`; usar store persistente.

• **M2 — Proteção de autenticação e autorização:** limitar tentativas por IP/usuário, adicionar proteção CSRF compatível com sessão e aplicar autorização por perfil/ação; manter mensagens de erro genéricas.

• **M3 — Minimização de dados e validação:** não retornar CPF completo por padrão (mascarar ou exigir permissão), validar dígitos verificadores, adicionar paginação e impor restrições únicas/índices apropriados no banco.

3. Modelo de dados

O DER mostra quatro tabelas. Clientes possuem vários veículos; veículos possuem vários agendamentos. Usuários são independentes.



PK = chave primária | FK = chave estrangeira | Usuários não possuem relação direta com as outras entidades.

4. API e front-end

• POST /api/login — recebe login e senha; 200 em sucesso, 400 se campos ausentes e 401 se credenciais inválidas.

• POST /api/logout — destrói a sessão.

• GET /api/me — retorna id e nome; exige sessão.

• GET/POST/PUT /api/clientes — lista/pesquisa, cria e edita clientes; exige sessão.

• GET/POST /api/veiculos — lista e cria veículos; exige sessão.

• GET/POST/PUT /api/agendamentos — lista, cria e altera data; exige sessão.

• Front-end: login.html/login.js; principal.html; clientes.html/clientes.js; veiculos.html/veiculos.js; agendamentos.html/agendamentos.js. comum.js centraliza chamadas, sessão expirada, escape HTML, formatação e modais.

5. Segurança

Sessão express-session com expiração de 30 minutos e cookie httpOnly. O segredo é fixo no código, cookie não define secure/sameSite, e o store padrão em memória não é indicado para produção. Senhas usam bcryptjs e comparação bcrypt.compareSync. CPF usa AES-256-GCM com IV aleatório e tag de autenticação, mas chave e salt são fixos no código. Consultas SQL dinâmicas usam parâmetros. Front-end escapa dados antes de interpolar HTML.

Três melhorias prioritárias

1. Mover segredo da sessão e chave/salt para variáveis de ambiente ou secret manager; rotacionar sessão no login; configurar secure/sameSite e store persistente. 2. Adicionar rate limit no login, proteção CSRF e autorização por papel. 3. Mascarar CPF nas respostas, validar dígitos verificadores e paginar listagens.

6. Casos de teste

ID	Caso	Resultado
T01	Inspeção do schema: tabela usuarios existe	PASS
T02	Inspeção do schema: tabela clientes existe	PASS
T03	Inspeção do schema: tabela veiculos existe	PASS

ID	Caso	Resultado
T04	Inspeção do schema: tabela agendamentos existe	PASS
T05	Verificar UNIQUE em usuarios.login	PASS
T06	Verificar UNIQUE em veiculos.placa	PASS
T07	Verificar FK veículos → clientes	PASS
T08	Verificar FK agendamentos → veículos	PASS
T09	Verificar bcrypt na autenticação	PASS
T10	Verificar proteção de rota sem sessão	PASS
T11	Verificar validação de CPF com 11 dígitos	PASS
T12	Verificar normalização/regex de placa	PASS
T13	Verificar validação de data real	PASS
T14	Requisição HTTP sem sessão a GET /api/me	BLOCKED
T15	Login válido admin/admin123 por HTTP	BLOCKED
T16	Login inválido retorna 401 por HTTP	BLOCKED
T17	Cadastro de cliente válido retorna 201	BLOCKED
T18	CPF duplicado retorna 400	BLOCKED
T19	Cadastro de veículo com placa duplicada retorna 400	BLOCKED
T20	Cadastro de agendamento com data inválida retorna 400	BLOCKED

Resultado real da execução

Node.js v22.16.0 foi detectado. A tentativa de iniciar o servidor falhou porque a dependência express não estava disponível (MODULE_NOT_FOUND). Assim, verificações estáticas foram marcadas PASS e os testes HTTP foram marcados BLOCKED; não há alegação falsa de que foram executados com sucesso. Consulte docs/evidencias/execucao-ambiente.txt.

7. Fluxo de cadastro de agendamento

```
Usuário → carrega veículos → sessão válida? → preenche veículo/data/serviço → POST /api/agendamentos → valida veículo →
```

8. Código comentado

A pasta sistema_comentado contém a aplicação original e comentários explicando os principais motivos das decisões de autenticação, validação, sessão, placa, data e criptografia. Os comentários não alteram a lógica.

Limitação de entrega

O ZIP não contém histórico Git. A pessoa estudante deve criar o repositório e distribuir pelo menos 8 commits reais em 3 dias diferentes. Não é possível produzir honestamente histórico de trabalho em dias distintos em uma única preparação.